

BẢN IN KỊCH BẢN CHUYỂN GIAO LIÊN TỤC (CD DEPLOYMENT SCRIPT)

Hệ thống Quản lý và Số hóa Tài liệu Thư viện HCMUS (HCMUS-LDMS)

THÔNG TIN TẬP NGUỒN (FILE CONTROL)

Trường thông tin	Nội dung chi tiết
Tên tập tin (File Name):	cd.yml
Đường dẫn trong Repo (Path):	.github/workflows/cd.yml
Công cụ CD áp dụng:	GitHub Actions
Mục đích:	Tự động hóa quy trình đóng gói container, kiểm tra hạ tầng staging/production, kích hoạt triển khai và gửi email Live URL qua Brevo
Sự kiện kích hoạt (Triggers):	Push vào nhánh main hoặc release/* sau khi vượt qua toàn bộ CI Pipeline

1. Giải thích quy trình Chuyển giao liên tục (CD Pipeline Flow)

- build-and-test**: Xác minh lần cuối toàn bộ kiểm thử trước khi tiến hành đóng gói phiên bản phát hành.
- deploy**: Đóng gói các Docker images (web, api), áp dụng biến môi trường Production an toàn, kiểm tra tính sẵn sàng của cơ sở dữ liệu PostgreSQL và kho MinIO.
- smoke-test**: Gửi yêu cầu HTTP kiểm tra endpoint GET /health đảm bảo hệ thống phản hồi mã trạng thái 200 OK.
- notify-deployment**: Tự động trích xuất Live URL triển khai và gửi thông báo nghiệm thu qua email Brevo.

2. Toàn văn mã nguồn kịch bản `.github/workflows/cd.yml`

```
name: CD

on:
  workflow_run:
    workflows: ["CI"]
    types: [completed]
  workflow_dispatch:
    inputs:
      environment:
        description: "Deployment Environment"
        required: true
        default: "production"
        type: choice
        options:
          - production
          - staging

jobs:
  build-verify:
    name: Build & Verify Artifacts
    if: github.event_name == 'workflow_dispatch' || github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${github.event.workflow_run.head_sha || github.sha}
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
      - name: Build frontend bundle
        working-directory: src/frontend
        run: |
          npm ci
          npm run build

  infra-verify:
    name: Verify Infrastructure (IaC)
    if: github.event_name == 'workflow_dispatch' || github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${github.event.workflow_run.head_sha || github.sha}
      - uses: hashicorp/setup-terraform@v3
      - name: Validate Infrastructure Blueprint
        working-directory: terraform
        run: |
          terraform fmt -check
          terraform init -backend=false
          terraform validate

  deploy:
    name: Deploy Application
    needs: [build-verify, infra-verify]
```

```

runs-on: ubuntu-latest
environment:
  name: ${GITHUB_EVENT.inputs.environment} || ((github.event.workflow_run.head_branch || git
  url: https://hcmus-projectmanage-lab.vercel.app
steps:
  - uses: actions/checkout@v4
    with:
      ref: ${GITHUB_EVENT.workflow_run.head_sha} || github.sha
  - name: Pull backend image from Docker Hub
    run: docker pull anhnguyen835/hcmus-ldms-api:${GITHUB_EVENT.workflow_run.head_sha} || gi
  - name: Deploy to Target Environment
    run: |
      ENV="${GITHUB_EVENT.inputs.environment} || ((github.event.workflow_run.head_branch ||
      echo "Deploying to environment: $ENV"
      echo "Frontend deployment URL: https://hcmus-projectmanage-lab.vercel.app"
      echo "Backend image pulled from Docker Hub; Render will auto-pull the updated ':latest'
  - name: Log image link
    run: |
      echo "### Backend image deployed" >> "$GITHUB_STEP_SUMMARY"
      echo "- Docker Hub: https://hub.docker.com/r/anhnguyen835/hcmus-ldms-api/tags" >> "$GIT
      echo "- Image: `anhnguyen835/hcmus-ldms-api:${GITHUB_EVENT.workflow_run.head_sha} ||

notify:
  name: Send CD Email Notification
  needs: [build-verify, infra-verify, deploy]
  if: always()
  runs-on: ubuntu-latest
  timeout-minutes: 5
  continue-on-error: true
  defaults:
    run:
      working-directory: src/backend
  steps:
    - uses: actions/checkout@v4
      with:
        ref: ${GITHUB_EVENT.workflow_run.head_sha} || github.sha
    - uses: astral-sh/setup-uv@v3
      with:
        version: "latest"
    - run: uv sync
    - name: Send deployment notification email
      env:
        BREVO_API_KEY: ${GITHUB_SECRETS.BREVO_API_KEY}
        BREVO_SENDER_EMAIL: ${GITHUB_SECRETS.BREVO_SENDER_EMAIL}
        DEPLOY_URL: "https://hcmus-projectmanage-lab.vercel.app"
        DEPLOY_ENV: ${GITHUB_EVENT.inputs.environment} || ((github.event.workflow_run.head_bra
        GITHUB_REPOSITORY: ${GITHUB_REPOSITORY}
        GITHUB_REF_NAME: ${GITHUB_EVENT.workflow_run.head_branch} || github.ref_name
        GITHUB_SHA: ${GITHUB_EVENT.workflow_run.head_sha} || github.sha
        GITHUB_SERVER_URL: ${GITHUB_SERVER_URL}
        GITHUB_RUN_ID: ${GITHUB_RUN_ID}
        COMMIT_MESSAGE: ${GITHUB_EVENT.workflow_run.head_commit.message} || github.event.head_
        COMMIT_AUTHOR: ${GITHUB_EVENT.workflow_run.head_commit.author.name} || github.event.he
        JOB_STATUSES: '{"build-verify": "${GITHUB_EVENT.workflow_run.head_commit.status}', "infra-verify": "${G
      run: uv run python -m app.scripts.send_deploy_notification

```

BẢN IN CẤU HÌNH HẠ TẦNG DOCKER COMPOSE PRODUCTION

Hệ thống Quản lý và Số hóa Tài liệu Thư viện HCMUS (HCMUS-LDMS)

THÔNG TIN TẬP NGUỒN (FILE CONTROL)

Trường thông tin	Nội dung chi tiết
Tên tập tin (File Name):	<code>docker-compose.prod.yml</code>
Đường dẫn trong Repo (Path):	<code>docker-compose.prod.yml</code>
Công nghệ:	Docker Compose v2 (Multi-container Orchestration)
Mục đích:	Định nghĩa toàn bộ hạ tầng vận hành: Reverse Proxy Nginx (TLS 8443), Frontend React, Backend FastAPI, CSDL PostgreSQL, Lưu trữ MinIO S3, Giám sát Prometheus & Grafana

1. Danh mục các dịch vụ cấu hình (Services Architecture)

- nginx**: Cổng đón tiếp HTTP (cổng 8080) và HTTPS (cổng 8443), định tuyến traffic tới Frontend và Backend.
- web**: Giao diện người dùng React/Next.js phục vụ độc giả và thủ thư.
- api**: Máy chủ backend FastAPI xử lý OCR, trích xuất metadata và quản lý phân quyền RBAC.
- postgres**: Cơ sở dữ liệu quan hệ PostgreSQL 16 lưu trữ người dùng, mục lục và siêu dữ liệu.
- minio**: Kho lưu trữ đối tượng S3-compatible lưu trữ tệp scan gốc và tệp sách điện tử EPUB.
- prometheus & grafana**: Hệ thống thu thập metrics và giám sát hiệu năng trực quan.

2. Toàn văn cấu hình `docker-compose.prod.yml`

```

services:
  web:
    build:
      context: .
      dockerfile: src/backend/nginx/Dockerfile
    ports:
      - "8080:80"
      - "8443:443"
    volumes:
      - ./src/backend/nginx/certs:/etc/nginx/certs:ro
    depends_on:
      - api
    restart: unless-stopped

  api:
    build:
      context: src/backend
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    environment:
      APP_ENV: ${APP_ENV:-production}
      DATABASE_URL: postgresql+psycpg://${POSTGRES_USER:-ldms}:${POSTGRES_PASSWORD:-ldms}@postgr
      MINIO_ENDPOINT: minio:9000
      MINIO_ACCESS_KEY: ${MINIO_ROOT_USER:-ldms}
      MINIO_SECRET_KEY: ${MINIO_ROOT_PASSWORD:-ldms12345}
      MINIO_BUCKET: ${MINIO_BUCKET:-ldms}
      MINIO_SECURE: ${MINIO_SECURE:-false}
      CORS_ORIGINS: '["http://localhost", "https://localhost", "http://localhost:8080", "https://
      OCR_DPI: ${OCR_DPI:-300}
      OCR_LANGUAGE: ${OCR_LANGUAGE:-vie+eng}
      OCR_TIMEOUT_SECONDS: ${OCR_TIMEOUT_SECONDS:-60}
      PDF_RENDER_TIMEOUT_SECONDS: ${PDF_RENDER_TIMEOUT_SECONDS:-120}
      PANDOC_TIMEOUT_SECONDS: ${PANDOC_TIMEOUT_SECONDS:-120}
      ENABLE MOCK_AUTH: ${ENABLE MOCK_AUTH:-true}
      JWT_SECRET: ${JWT_SECRET:-dev-insecure-secret-change-me-min-32-bytes}
      JWT_ALGORITHM: ${JWT_ALGORITHM:-HS256}
      JWT_EXPIRES_MINUTES: ${JWT_EXPIRES_MINUTES:-1440}
      GOOGLE_CLIENT_ID: ${GOOGLE_CLIENT_ID:-}
      GOOGLE_CLIENT_SECRET: ${GOOGLE_CLIENT_SECRET:-}
      GOOGLE_REDIRECT_URI: ${GOOGLE_REDIRECT_URI:-}
      GOOGLE_ALLOWED_DOMAINS: ${GOOGLE_ALLOWED_DOMAINS:-[]}
      FRONTEND_BASE_URL: ${FRONTEND_BASE_URL:-http://localhost:8080}
      SMTP_HOST: ${SMTP_HOST:-mailhog}
      SMTP_PORT: ${SMTP_PORT:-1025}
      AUTO_SEED_DATA: ${AUTO_SEED_DATA:-true}
    depends_on:
      postgres:
        condition: service_healthy
      minio:
        condition: service_healthy
    restart: unless-stopped

  postgres:

```

```

image: postgres:16
environment:
  POSTGRES_USER: ${POSTGRES_USER:-ldms}
  POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-ldms}
  POSTGRES_DB: ${POSTGRES_DB:-ldms}
ports:
  - "5434:5432"
volumes:
  - postgres_prod_data:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-ldms}"]
  interval: 5s
  timeout: 5s
  retries: 5
restart: unless-stopped

minio:
image: minio/minio:latest
command: server /data --console-address ":9001"
environment:
  MINIO_ROOT_USER: ${MINIO_ROOT_USER:-ldms}
  MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD:-ldms12345}
ports:
  - "9002:9000"
  - "9003:9001"
volumes:
  - minio_prod_data:/data
healthcheck:
  test: ["CMD", "mc", "ready", "local"]
  interval: 5s
  timeout: 5s
  retries: 5
restart: unless-stopped

mailhog:
image: mailhog/mailhog:latest
ports:
  - "1025:1025"
  - "8025:8025"
restart: unless-stopped

prometheus:
image: prom/prometheus:latest
volumes:
  - ./monitoring/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
ports:
  - "9090:9090"
restart: unless-stopped

grafana:
image: grafana/grafana:latest
environment:
  - GF_SECURITY_ADMIN_USER=admin
  - GF_SECURITY_ADMIN_PASSWORD=admin
  - GF_USERS_ALLOW_SIGN_UP=false
volumes:
  - ./monitoring/grafana/provisioning/datasources:/etc/grafana/provisioning/datasources:ro

```

```
- ./monitoring/grafana/provisioning/dashboards:/etc/grafana/provisioning/dashboards:ro
- ./monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards/json:ro
- grafana_prod_data:/var/lib/grafana
ports:
  - "3000:3000"
depends_on:
  - prometheus
restart: unless-stopped

volumes:
  postgres_prod_data:
  minio_prod_data:
  grafana_prod_data:
```

HƯỚNG DẪN TRIỂN KHAI VÀ VẬN HÀNH

Hệ thống Quản lý và Số hóa Tài liệu Thư viện HCMUS (HCMUS-LDMS)

THÔNG TIN TÀI LIỆU (DOCUMENT CONTROL)

Trường thông tin (Field)	Nội dung đặc tả (Description)
Mã tài liệu (Document ID)	HCMUS-LDMS-DEPLOY
Tên tài liệu (Document Title)	Hướng dẫn Triển khai và Vận hành (Deployment Guide)
Dự án (Project Name)	HCMUS-LDMS
Đơn vị soạn thảo (Author/Organization)	Nhóm Phát triển Dự án HCMUS-LDMS
Đối tượng đọc	Kỹ sư vận hành / DevOps thành viên nhóm
Cấp độ bảo mật (Security Class)	Internal
Trạng thái tài liệu (Status)	Active — Continuous Delivery thủ công qua <code>scripts/run-prod.sh</code>

LỊCH SỬ PHIÊN BẢN (REVISION HISTORY)

Phiên bản (Version)	Ngày phát hành (Date)	Mô tả thay đổi (Description of Change)	Người thực hiện (Author)
1.0	20/08/2026	Khởi tạo Deployment Guide (bản nháp gần script tạm).	Nguyễn Tuấn Anh
1.1	20/08/2026	Đồng bộ main: dùng <code>scripts/run-prod.sh</code> + <code>docker-compose.prod.yml</code> (Web, MailHog, Prometheus, Grafana); bỏ tham chiếu <code>deploy-prod.sh</code> .	Nguyễn Tuấn Anh
1.2	22/08/2026	Bổ sung workflow <code>.github/workflows/cd.yml</code> tự động build/deploy, xuất URL live và gửi email Brevo qua job notify.	Ân Tiến Nguyễn An
1.3	22/08/2026	Bổ sung mô-đun Infrastructure as Code (IaC) dùng Terraform (<code>terraform/</code>) hoàn thiện bộ 3 DevOps (IaC + CI + CD).	Ân Tiến Nguyễn An

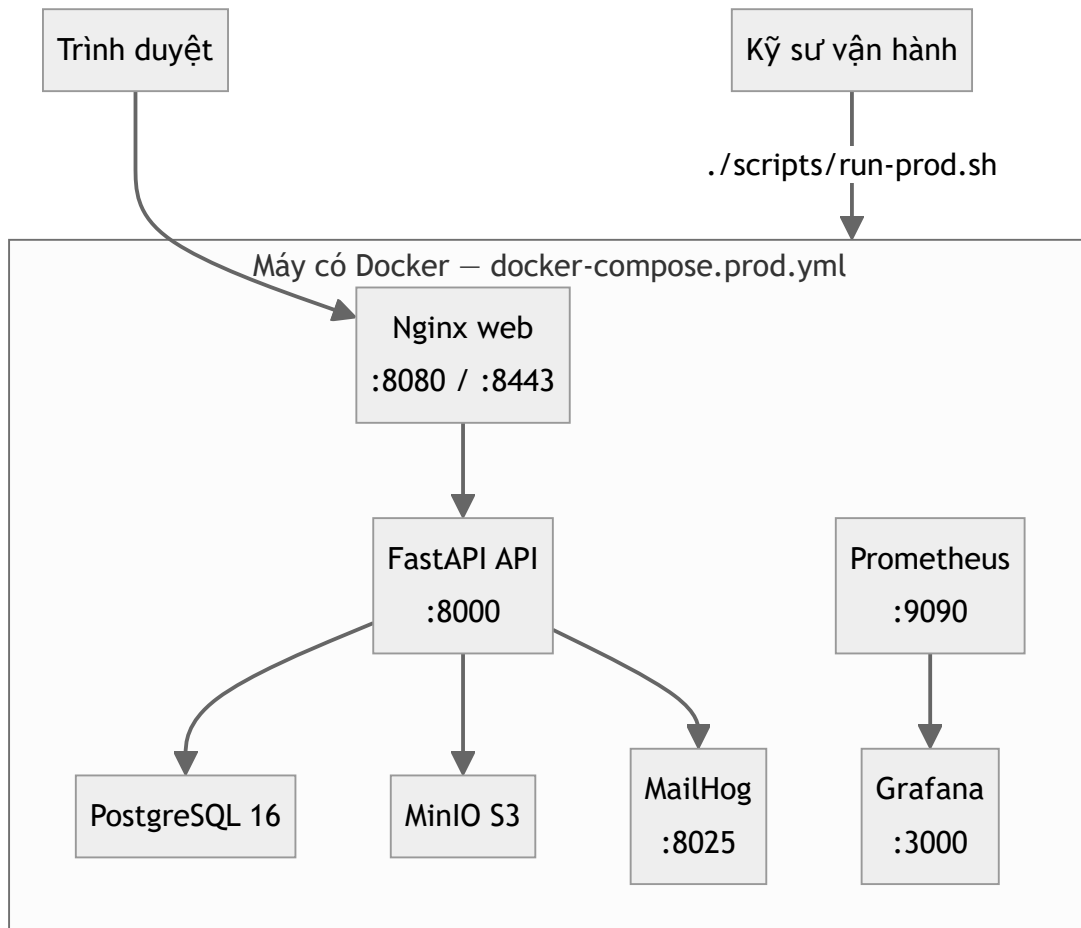
1. Mục đích và Phạm vi thực tế

Tài liệu hướng dẫn **triển khai tái lập được** stack HCMUS-LDMS bằng Terraform IaC, Docker Compose và pipeline GitHub Actions CI/CD trong repo.

Đã có trong repo	Chưa có (gap / roadmap)
<code>scripts/run.sh</code> — one-shot dev	Tự động provisioning hạ tầng VMware
<code>scripts/run-prod.sh</code> + <code>docker-compose.prod.yml</code> — prod one-click	PgBackRest / Restic đầy đủ như architecture \$5.2
<code>terraform/</code> (<code>main.tf</code> , <code>variables.tf</code> , <code>outputs.tf</code>) — IaC declarative stack	—
<code>.github/workflows/cd.yml</code> — CD pipeline (Build, Deploy, URL output & Email notify)	—
Health check + seed demo sau khi API lên	—
Prometheus + Grafana + MailHog trong stack prod	—
<code>backup-postgres.sh</code> , <code>backup-minio.sh</code>	—

Theo tài liệu lý thuyết: nhóm đáp ứng trọn vẹn mô hình $\text{DevOps} = \text{IaC} + \text{CI} + \text{CD}$ qua Terraform (`terraform/`), GitHub Actions CI (`ci.yml`), GitHub Actions CD (`cd.yml`), và triển khai one-click tại chỗ (`./scripts/run-prod.sh`).

2. Kiến trúc triển khai ngắn



Chi tiết C4 Deployment View (VM Staging/Prod mục tiêu trên VMware): `02-architecture.md` §6.

File	Vai trò
<code>docker-compose.prod.yml</code> (gốc repo)	Stack production-like đầy đủ
<code>src/backend/docker-compose.yml</code>	Stack dev (API + Postgres + MinIO; Nginx qua profile prod cũ nếu còn dùng)
<code>scripts/run-prod.sh</code> / <code>scripts/run-prod.ps1</code>	One-click prod: build/up → health → seed

Services trong `docker-compose.prod.yml`

Service	Vai trò
<code>web</code>	Nginx — FE tĩnh + TLS, cổng 8080/8443
<code>api</code>	FastAPI (<code>APP_ENV=production</code> mặc định)
<code>postgres</code>	PostgreSQL 16 (volume <code>postgres_prod_data</code>)
<code>minio</code>	Object storage (volume <code>minio_prod_data</code>)
<code>mailhog</code>	SMTP mock + UI <code>:8025</code>
<code>prometheus</code>	Metrics <code>:9090</code>
<code>grafana</code>	Dashboard <code>:3000</code> (admin/admin mặc định)

3. Triển khai môi trường Development

Chi tiết: [06-developer-guide.md](#).

```
./scripts/run.sh
```

Health: `http://localhost:8000/health` → sau đó Alembic `upgrade head` (trong `run.sh`).

4. Triển khai Production-like (one-click)

Mục tiêu: bài toán Deployment script buổi 08 — **chạy trên máy bất kỳ có Docker** → toàn bộ hệ thống lên (Web, API, DB, storage, mail, monitoring).

4.1 Chuẩn bị

```
# Checkout đúng tag/commit cần phát hành
# Kiểm tra biến môi trường (file .env / export) trước khi coi là production thật:
#   JWT_SECRET mạnh, ENABLE MOCK_AUTH=false nếu cần, OAuth Google, v.v.
```

Script tự tạo cert self-signed vào `src/backend/nginx/certs/` nếu thiếu `dev.crt` / `dev.key` (cần `openssl`).
Cert này **chỉ demo** — máy trường cần CA hợp lệ.

4.2 Chạy script chính thức

```
./scripts/run-prod.sh
# Windows: scripts/run-prod.ps1
```

Luồng thực tế trong script:

1. Kiểm tra Docker daemon + Compose plugin.
2. Sinh TLS self-signed nếu thiếu.
3. `docker compose -f docker-compose.prod.yml up -d --build`.
4. Chờ tối đa ~60s tới khi `GET http://localhost:8000/health` thành công; fail → in log `api` và exit $\neq 0$.
5. Chạy seed: `uv run python -m app.scripts.seed_data` trong container `api` (cảnh báo nếu đã seed sẵn).

Dừng stack:

```
docker compose -f docker-compose.prod.yml down
```

4.3 URL sau khi lên (theo output script)

URL	Ý nghĩa
<code>http://localhost:8080</code>	Web (HTTP)
<code>https://localhost:8443</code>	Web (HTTPS)
<code>http://localhost:8000</code>	API
<code>http://localhost:8000/docs</code>	OpenAPI
<code>http://localhost:9003</code>	MinIO Console
<code>http://localhost:8025</code>	MailHog
<code>http://localhost:3000</code>	Grafana
<code>http://localhost:9090</code>	Prometheus

Tài khoản demo (seed): xem banner cuối `run-prod.sh` (admin / librarian / reader `@hcmus.edu.vn`).

4.4 Rollback khi lỗi (thực hành MVP)

Chưa có registry image versioning đầy đủ:

```
docker compose -f docker-compose.prod.yml down
git checkout <commit-hoặc-tag-trước-đó>
./scripts/run-prod.sh
# Khôi phục dữ liệu nếu cần: pg_restore / mirror MinIO từ backup (mục 6)
```

4.5 Khởi tạo hạ tầng bằng Terraform IaC (`terraform/`)

Để đảm bảo nguyên lý **Infrastructure as Code (IaC)** chuẩn hóa theo công thức $\text{DevOps} = \text{IaC} + \text{CI} + \text{CD}$, nhóm cung cấp cấu hình Terraform khai báo toàn bộ tài nguyên:

1. Khởi tạo và tải Docker provider:

```
bash cd terraform terraform init
```

2. Kiểm tra kế hoạch hạ tầng (Execution Plan):

```
bash terraform plan
```

Output hiển thị 7 tài nguyên được khởi tạo tự động (`docker_network` , `docker_volume` , PostgreSQL, MinIO, MailHog, Prometheus, Grafana).

3. Áp dụng hạ tầng:

```
bash terraform apply -auto-approve
```

4. Xem các endpoint đầu ra:

```
bash terraform output
```

Output xuất Live URL, API Docs URL (`:8000/docs`), MinIO Console (`:9003`), Grafana Dashboard (`:3000`).

5. Dọn dẹp hạ tầng khi kết thúc:

```
bash terraform destroy -auto-approve
```

5. Cấu hình CSDL

- Engine: **PostgreSQL 16** (service `postgres` trong compose prod/dev).
- Ứng dụng: SQLAlchemy + Alembic cho schema; `run.sh` (dev) gọi `alembic upgrade head` tường minh.
- `run-prod.sh` sau health chạy **seed dữ liệu demo** (`app.scripts.seed_data`). Khi phát hành schema mới trên môi trường đã có data, vận hành viên vẫn nên xác nhận migration Alembic đã áp dụng (theo quy trình dev / README module backend).

Biến kết nối: `DATABASE_URL=postgresql+psycopg:// ... @postgres:5432/ ...`.

6. Sao lưu và Khôi phục (MVP)

MVP thay thế PgBackRest/Restic (architecture §5.2) — xem thêm root README nếu còn mục backup.

```
./scripts/backup-postgres.sh # pg_dump -Fc → .backups/postgres/
./scripts/backup-minio.sh   # mc mirror → .backups/minio/
```

RPO/RTO thiết kế (architecture §5.3): RPO ≤ 24h, RTO ≤ 4h — phụ thuộc tần suất chạy script khi vận hành thật.

7. Multi-environment

Môi trường	Cách nhóm hiện thực	Ghi chú
Dev	<code>scripts/run.sh</code> + <code>src/backend/docker-compose.yml</code> + Vite	Mock auth có thể bật
Prod-like (local / demo máy bất kỳ)	<code>scripts/run-prod.sh</code> + <code>docker-compose.prod.yml</code>	Đủ Web, mail, Grafana/Prometheus
Staging / Production VMware	Deployment View architecture	Chưa gắn CD pipeline trong repo

Không dùng chung secret yếu / mock auth của dev trên máy coi là production.

8. Giám sát (Monitor) & Thông báo

Hạng mục	Hiện trạng
Health check lúc deploy	Có — <code>/health</code> trong <code>run.sh</code> và <code>run-prod.sh</code>
Email sau merge <code>main</code> (CI)	Có — Brevo qua job CI <code>notify</code> (<code>send_merge_notification.py</code>)
Email sau deploy (CD)	Có — Brevo qua job CD <code>notify</code> (<code>send_deploy_notification.py</code>) với Live URL
Prometheus + Grafana	Có trong stack prod (<code>docker-compose.prod.yml</code>)
Alert SLA / on-call đầy đủ	Tùy cấu hình dashboard — chưa mô tả runbook PagerDuty

9. Checklist vận hành trước demo / phát hành thử công

- [] CI trên commit/tag định phát hành đang **xanh**.
- [] Secret / `JWT_SECRET` / mock auth đã rà soát nếu demo gần production.
- [] `./scripts/run-prod.sh` thành công; mở được `https://localhost:8443` (hoặc URL máy chủ).
- [] Grafana `:3000` / Prometheus `:9090` phản hồi (xác nhận Monitor).
- [] Đã chạy backup trước thao tác rủi ro trên dữ liệu thật.
- [] Ghi nhận tag Git (ví dụ `v1.0.0-MVP`) trên commit đã deploy.

10. Tài liệu liên quan

- [06-developer-guide.md](#)
- [09-test-plan.md](#)
- [scripts/run-prod.sh](#)
- [docker-compose.prod.yml](#)
- [terraform/](#) — Cấu hình Terraform IaC ([main.tf](#) , [variables.tf](#) , [outputs.tf](#))
- [.github/workflows/ci.yml](#) — CI (tích hợp liên tục)
- [.github/workflows/cd.yml](#) — CD (chuyển giao liên tục + email notify)

← CD

Merge pull request #54 from se-bros/feat/iaac-terraform-evidence #2

Re-run all jobs

Summary

All jobs

- Build & Verify Artifacts
- Verify Infrastructure (IaaC)
- Deploy Application
- Send CD Email Notification

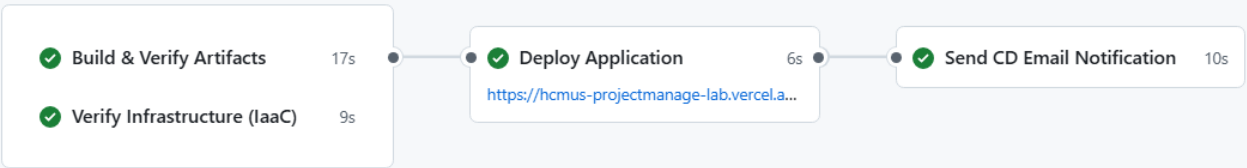
Run details

- Usage
- Workflow file

Triggered via push 1 minute ago

Status	Total duration	Artifacts
Success	40s	-

cd.yml
on: push



Annotations

4 warnings

- Verify Infrastructure (IaaC)

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, hashicorp/setup-terraform@v3. For more inf...

Show more
- Build & Verify Artifacts

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-node@v4. For more informatio...

Show more
- Deploy Application

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: https://github.h...

[se-bros/hcmus-projectmanage--lab]  CD (production) - main: Merge pull request #54 from se-bros/feat/iaac-terraform-evidence  Summarize this email



tuananh835.nta@11103689.brevo.send.com

To: tuananh835.nta@gmail.com;  AN TIẾN NGUYỄN AN; anhkhoea515.dev@gmail.com; +2 others

Sat 8/22/2026 2:06 PM

 Some content in this message has been blocked because the sender isn't in your Safe senders list.

Trust sender

Show blocked content

se-bros/hcmus-projectmanage--lab vừa triển khai lên môi trường **PRODUCTION**.

 **Live URL:** <https://hcmus-projectmanage-lab.vercel.app>

- Branch / Tag: main
- Environment: production
- Commit: [a173a03](#)
- Author: Nguyen An
- Message: Merge pull request #54 from se-bros/feat/iaac-terraform-evidence Feat/iaac terraform evidence
- Workflow run: <https://github.com/se-bros/hcmus-projectmanage--lab/actions/runs/32558692105>
- Job run:
 - build-verify:  success
 - infra-verify:  success
 - deploy:  success
- Status:  success

 Reply

 Reply all

 Forward

[se-bros/hcmus-projectmanage--lab] ✅ CD (production) - main: Merge pull request #52 from se-bros/lab06-final

 Summarize this email



tuananh835.nta@11103689.brevosend.com

To: tuananh835.nta@gmail.com; ⓘ AN TIẾN NGUYỄN AN; anhkhoa515.dev@gmail.com; +2 others

Sat 8/22/2026 11:10 AM



ⓘ Some content in this message has been blocked because the sender isn't in your Safe senders list.

Trust sender

Show blocked content

se-bros/hcmus-projectmanage--lab vừa triển khai lên môi trường **PRODUCTION**.

🌐 **Live URL:** <https://hcmus-projectmanage-lab.vercel.app>

- Branch / Tag: main
- Environment: production
- Commit: [c636ba5](#)
- Author: Nguyen An
- Message: Merge pull request #52 from se-bros/lab06-final Lab06 final: check CD's output with sending email with URL
- Workflow run: <https://github.com/se-bros/hcmus-projectmanage--lab/actions/runs/32550946327>
- Job run:
 - build-verify: ✅ success
 - deploy: ✅ success
- Status: ✅ success

 Reply

 Reply all

 Forward

< CD

Merge pull request #52 from se-bros/lab06-final #1

Re-run all jobs

Summary

All jobs

Build & Verify Artifacts

Deploy Application

Send CD Email Notification

Run details

Usage

Workflow file

Triggered via push 2 hours ago	Status	Total duration	Artifacts
NguyenAn0808 pushed c636ba5 main	Success	42s	—

cd.yml

on: push

Build & Verify Artifacts

19s

Deploy Application

6s

<https://hcmus-projectmanage-lab.vercel.a...>

Send CD Email Notification

8s

Annotations

3 warnings

Build & Verify Artifacts

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-node@v4. For more informatio...
[Show more](#)

Deploy Application

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: <https://github.b...>
[Show more](#)

Send CD Email Notification

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, astral-sh/setup-uv@v3. For more informatio...
[Show more](#)